

A Project Report
On
NOVEL SORTING ALGORITHMS

BY

**N.KUSHWANTH – SE22UCSE177
M.PANVI TEJ – SE22UCSE194
ARYAN .C – SE22UCSE035
E.ANVITH TEJ – SE22UCSE089
B.JATHIN REDDY–SE22UCSE118
A.HEMANTH – SE22UCSE109
P.NIHAL RAO – SE22UCSE181
P.RITHVIK RAO – SEE2UCSE210**

**Under the supervision of
DR.GARIMELLA RAMA MURTHY SIR**

**SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF
PR 4204: 8th semester Project**



**MAHINDRA ECOLE CENTRALE
COLLEGE OF ENGINEERING
HYDERABAD
(December 2025)**

ACKNOWLEDGMENTS

We would like to express our gratitude to our project supervisor, **Dr. Garimella Rama Murthy sir**, Professor, Department of Computer Science and Engineering for his advice and continuous support. His expertise and mentorship played a crucial role in our project's success. We are greatly thankful for his dedication and patience in guiding us through the challenges and providing valuable feedback that helped improve our work.

We would like to acknowledge our sincere appreciation for the knowledge and skills we have gained throughout this project. The experience of working on this project has been valuable for our upcoming future, and we are grateful for the opportunity to apply the concepts we have learned in a practical setting.



**Mahindra Ecole Centrale,
College of Engineering, Mahindra University
Hyderabad**

Certificate

This is to certify that the project report entitled “Novel Sorting Algorithms (Boolean threshold decomposition & Weighted Median Applications)” submitted by

N.Kushwanth SE22UCSE177 , M.Panvi Tej SE22UCSE194 , Aryan .c SE22UCSE035
P.Nihal SE22UCSE181 , E.Anvith Tej SE22UCSE089 , B.Jathin Reddy SE22UCSE118 ,
A.Hemanth SE22UCSE109 , P.Rithvik Rao SE22UCSE210 in partial fulfillment of the
requirements of the course PR4204 – Project Course embodies the work carried out under my
supervision and guidance.

(SUPERVISOR NAME & Signature)

Mahindra Ecole Centrale, Hyderabad.

Date:

ABSTRACT

This work presents a novel threshold-based computational framework that bridges concepts from Boolean mathematics, signal processing. The core idea is to transform numerical data into a threshold Boolean representation, where each value is evaluated across multiple threshold levels. This conversion creates a structured binary matrix that preserves the relative ordering and distribution of the original data while enabling efficient logical operations.

Building on this representation, the project introduces the concept of weighted signal samples. Instead of treating all samples equally, each threshold level contributes a weighted amount of information,. This weighted threshold mechanism provides a compact way to extract statistical properties such as minimum, maximum, median, ranking, sorting behavior, and distribution characteristics without repeatedly scanning the original dataset.

From a signal processing perspective, the threshold matrix can be viewed as a multi-level signal decomposition, where each threshold acts as a filtering stage. The resulting weighted threshold sums form a signal signature that can be analyzed to identify patterns, trends, and structural properties within the data. This approach enables efficient feature extraction while reducing computational complexity.

A major contribution of this work is the study of weighted signal samples and weighted median filtering through Boolean formulations. We show the relation between when signal sample weights are equal,unequal case the Boolean expression determining the filter output to that of the ordinary median filter. This demonstrates that the proposed framework naturally generalizes classical median filtering without increasing computational complexity.

Overall, the proposed framework offers a unified methodology for data representation, signal analysis, sorting, statistical computation, and distributed learning. By combining Boolean threshold structures, weighted signal samples, signal processing principles, and federated learning, the work demonstrates a scalable and privacy-conscious approach for handling complex datasets across a wide range of real-world applications, including IoT systems, healthcare monitoring, sensor networks, edge computing, and large-scale data analytics

INDEX	
Title Page	1
Acknowledgements	2
Certificate	3
Abstract	4
Introduction	6
2.1 Boolean Matrix Construction (Unweighted)	6
2.2 Threshold Decomposition	7
2.3 Example (Unweighted)	8
Problem Definition(Chapter 3)	8
Weighted Median Algorithm	9
Complexity	11
Background and Related Work (Chapter 4)	12
Boolean Expressions	14
4.1 Weighted Median as a Boolean Expression	15
4.2 Equal Weights Case Expression	15
4.3 Unequal Weights Case Expression	16
Implementation (Chapter 5)	17
5.1 Implementation Strategy	17
5.2 Tools and Technologies Used	17
5.3 Datasets Used	17
Image Denoising	18
Signal Denoising	18
Byzantine Robust Aggregation in Federated Learning	20
5.4 Testing and Validation	20
5.5 Project Timeline	20
Results and Analyses (Chapter 6)	21
7)Conclusion	23
8)References	24

Chapter 2 : :

Introduction

Modern AI and signal-processing systems are becoming increasingly data-intensive. Whether processing medical images, sensor streams, network traffic, or edge-device data, current approaches often require repeated sorting, complex comparisons, and significant computational resources. These operations become particularly challenging in real-time environments where speed, scalability, and privacy are critical.

Our project introduces a novel threshold-based computational framework that transforms numerical data into a structured Boolean representation. Instead of repeatedly manipulating raw values, the system converts data into threshold layers, enabling efficient extraction of statistical and decision-making information through simple logical operations and Boolean expressions.

The core innovation is the conversion of numerical arrays into a Boolean matrix, where each threshold level represents a logical view of the data. Through carefully designed Boolean expressions, the framework can analyze patterns, perform comparisons, and extract meaningful information without relying on traditional sorting procedures. This allows important quantities such as maximum values, medians, weighted medians, distributions, and ranking information to be determined using logical computation rather than computationally expensive arithmetic operations.

The concept extends naturally to weighted signal samples, where each sample contributes according to its significance. By combining threshold representations with Boolean expressions, the system can efficiently process large-scale signals and identify critical information in real time. This creates a unified framework that connects data representation, logical reasoning, and statistical analysis.

The potential applications are extensive. In image processing, Boolean threshold layers can be used for image denoising, segmentation, feature extraction, and intensity analysis. In signal processing, the framework enables efficient handling of weighted sensor measurements and streaming data. In federated learning, local devices can compute Boolean-based summaries and decision metrics without sharing raw data, reducing communication overhead while enhancing privacy preservation.

Unlike conventional approaches that depend heavily on sorting and repeated numerical operations, our framework leverages Boolean expressions as a computational engine. This shift transforms complex analytical problems into logical operations that are easier to execute, scale, and distribute across devices.

By integrating Boolean algebra, weighted signal processing, image analysis, and federated learning into a single computational model, this project presents a new paradigm for efficient data processing. It demonstrates how logical representations can simplify complex computations, reduce resource consumption, and support the development of faster, scalable, and privacy-aware intelligent systems for future real-world applications.. This creates a compact and information-rich representation that enables efficient sorting, ranking, statistical analysis, and pattern discovery.

Representing this project in simplest mathematical format core idea is to represent an integer array $a = [a_1, a_2, \dots, a_N]$ as a Boolean matrix. For $MAX = \max a_i$, a matrix of size $(MAX + 1) \times (N + 1)$ is constructed where:

- The first row stores the input elements.
- The first column stores thresholds from 0 to MAX.
- Entry $(t, i) = 1$ if $a_i \geq t$, else 0.
-

From this 0/1 matrix, median, are computed using row-wise counts and a decision column V .
Boolean expressions using AND/OR gates are derived for these statistics.

Boolean Matrix Construction (Unweighted)

1. Threshold Decomposition

For array $a = [a_1, \dots, a_N]$ and threshold t :

$$X_i(t) = \begin{cases} 1, & a_i \geq t \\ 0, & a_i < t \end{cases}$$

Let $\text{ones}(t) = \sum X_i(t)$, $\text{zeros}(t) = N - \text{ones}(t)$.

Step : Iterate Over Thresholds with Width

1 Input

- Array $a = [a_1, \dots, a_N]$.

1. Let T be the list of distinct threshold values in a :

We maintain accumulators:

median = 0, and

a variable prev , initially 0.

For each threshold $\tau_k \in T$:

1. Let $\text{width} = \tau_k - \text{prev}$ and then set $\text{prev} \leftarrow \tau_k$.

- Compute Boolean row $X(\tau_k)$ and ones, zeros
- Decision variables:

$$V_{\text{med}}(\tau_k) = \begin{cases} 1, & \text{ones} > \text{zeros} \\ 0, & \text{otherwise} \end{cases}$$

(here we consider median because it can be suited for making threshold decision.

EXAMPLE

$$a = [3, 5, 1, 6, 9],$$

$T = [3, 5, 1, 6, 9]$. here $X_1, X_2, \dots$ are actually input values

The Boolean rows for each threshold are:

t	X_1	X_2	X_3	X_4	X_5	(ones, zeros)
1	1	1	1	1	1	(5, 0)
3	0	1	1	1	1	(4, 1)
5	0	0	1	1	1	(3, 2)
6	0	0	0	1	1	(2, 3)
9	0	0	0	0	1	(1, 4)

(Note: no need to put in ascending order form, here this is for easier representation)

median = 5. here the no of ones just crossed half ≥ 3 (5+1/2)

CHAPTER 3

PROBLEM DEFINITION

Computing robust statistical measures such as the median and weighted median can be computationally expensive for large-scale data. This project proposes a threshold-based Boolean computational framework representing numerical data as Boolean matrices and performing aggregation through logical operations. The approach enables efficient processing of signals, images, and distributed learning updates while maintaining robustness against noise and outliers.

1) Other than threshold based decision making can it also be used for arithmetic ranking?

Let Although it is not the main aspect this can be used for reranking

After the threshold take $p_{\max} = 0, \quad p_{\min} = 0,$

$$V_{p_{\max}}(\tau_k) = \begin{cases} 1, & \text{ones} \geq P_{\max} \\ 0, & \text{otherwise} \end{cases}$$

$$V_{p_{\min}}(\tau_k) = \begin{cases} 1, & \text{zeros} < P_{\min} \\ 0, & \text{otherwise} \end{cases}$$

$$p_{\max} += \text{width} \cdot V_{p_{\max}}(\tau_k), \quad p_{\min} += \text{width} \cdot V_{p_{\min}}(\tau_k).$$

Apply these in the boolean matrix and these can be used for ranking of maximum and minimum

2) What will you do if there are weights involved in the signal samples?

Weighted Median Algorithm

2.1 Mathematical Explanation

The weighted median is the smallest value such that the cumulative weight of all elements not exceeding it is at least half of the total weight.

In our implementation, we:

1. Compute the cumulative sum of weights.
2. The value where the cumulative weight first exceeds half of the total weight is declared the weighted median.

This guarantees that at least 50% of the weight lies on or below the median value, which matches the mathematical definition of the weighted median.

2.2 Formal Setup

We have:

$$a = [a_1, a_2, \dots, a_N], \quad w = [w_1, w_2, \dots, w_N],$$

where each $w_i > 0$ indicates the importance or frequency of a_i .

Total Weight and Half Threshold

We take the total weight as W_{tot}
half-weight threshold is

$$T = \frac{W_{\text{tot}}}{2}.$$

In practice, the condition is sufficient to locate the weighted median index k : the smallest k such that $C_k \geq \frac{W_{\text{tot}}}{2}$.

2.3 Numerical Example

Consider:

$$a = [4, 5, 6, 7, 8], \quad w = [3, 2, 4, 6, 5] \quad W_{\text{tot}} = 20$$

The cumulative weights from the bottom are:

$$C = [3, 5, 9, 15, 20].$$

$a = [4, 5, 6, 7, 8], w = [3, 2, 4, 6, 5], W_{\text{tot}} = 20, \text{half} = 10$
Cumulative: $[3, 5, 9, 15, 20] \rightarrow \text{Weighted Median} = 7$

3) How will the threshold decision for signal samples be affected if all the signal samples are of equal weights?

The ordinary median of this expanded multiset is exactly the weighted median of the original set. The same Boolean expressions for the median (using replicated Boolean variables) can be applied, and this is consistent with the cumulative-weight method given above.

When all weights are equal, each element is replicated the same number of times, and hence the weighted median coincides with the ordinary median.

Complete Example: Unweighted vs Equal Weights

We now provide a detailed Boolean matrix construction for both the unweighted and equal-weighted median for the same dataset:

$$a = [1, 2, 3, 4, 5], \quad w = [2, 2, 2, 2, 2].$$

Thus, every element is repeated twice in the conceptual multiset, and therefore the weighted median must match the standard median. We now prove this using the Boolean threshold decomposition structure.

1: Boolean Threshold Matrix (Unweighted)

Here the rule for each threshold t is:

$$X_i(t) = \begin{cases} 1, & a_i \geq t, \\ 0, & a_i < t. \end{cases}$$

The threshold values are the input values themselves:

$$T = \{1, 2, 3, 4, 5\}.$$

t	X_1	X_2	X_3	X_4	X_5	#1's	V_{med}
1	1	1	1	1	1	5	1
2	0	1	1	1	1	4	1
3	0	0	1	1	1	3	1
4	0	0	0	1	1	2	0
5	0	0	0	0	1	1	0

Table 4.1: Unweighted Boolean threshold matrix.

Decision rule:

$$V_{\text{med}}(t) = \begin{cases} 1, & \#1\text{'s} \geq \lceil N/2 \rceil = 3, \\ 0, & \text{otherwise.} \end{cases}$$

Thus:

$$\text{Median} = \sum V_{\text{med}}(t) = 1 + 1 + 1 + 0 + 0 = 3.$$

Step 2) weighted median via cumulative weights for equal weights

Since each a_i has weight 2:

Weight below/equal to $t = 2 \cdot (\# \text{ elements } \leq t)$.

t	#elements $\leq t$	Weight $\leq t$	V_{wmed}
1	1	2	0
2	2	4	0
3	3	6 (≥ 5)	1
4	4	8	1
5	5	10	1

Table 4.2: Weighted threshold decision for equal weights.

Thus:

$$\text{Weighted Median} = \min\{t \mid V_{\text{wmed}}(t) = 1\} = 3.$$

Conclusion of the Example

Both unweighted and weighted Boolean formulations yield:

$$\text{Median} = \text{Weighted Median} = 3$$

because equal weights replicate data without altering its center. Therefore, the Boolean median formula remains unchanged when all weights are identical.

4) What is reduced complexity of this algorithm how is it better than traditional threshold based algorithm Complexity

Let R be the number of distinct values in the array.

- i. Time complexity: $O(NR)$, since we scan all N elements for each of the R thresholds.
- ii. Space complexity: $O(N)$ for each threshold row (or $O(NR)$ if the entire matrix is stored explicitly).

This improves upon the original $O(N \cdot \text{MAX})$ when $R \ll \text{MAX}$

CHAPTER 4

Background and Related Work

This section reviews foundational and recent works relevant to Boolean-based sorting, order-statistic filtering, signal denoising, and federated learning aggregation. The literature is organized thematically.

Classical Sorting and Order Statistics

[1] Knuth (1998) — The Art of Computer Programming, Vol. 3: Sorting and Searching provides the canonical reference for comparison-based sorting. Knuth establishes the $\Omega(N \log N)$ lower bound for comparison-based sorting and catalogs algorithms such as MergeSort and HeapSort. While theoretically optimal, these approaches require repeated element comparisons over raw values, making them expensive in hardware implementations and distributed settings. This work motivates the search for alternative, logic-based approaches that bypass explicit comparisons.

[2] Blum, Floyd, Pratt, Rivest, and Tarjan (1973) — 'Time Bounds for Selection' introduced the linear-time median selection algorithm (the median-of-medians). While a landmark theoretical result, its practical overhead makes it less suited for real-time or hardware-constrained environments. Our Boolean threshold approach achieves $O(NR)$ complexity in a structurally different way, with the added benefit of admitting bit-parallel hardware implementations.

[3] Sun, Ye, Sun, and Kameda (2015) — 'Improved Algorithms for Exact and Approximate Boolean Matrix Decomposition' (IEEE TKDE) addresses the problem of decomposing a binary matrix into a product of two smaller Boolean matrices. While their focus is on matrix factorization for knowledge discovery, their algorithmic insights on exploiting distinct row/column patterns to reduce computation parallel the threshold-grouping optimization in our framework.

[4] Parhami (2010) — Computer Arithmetic: Algorithms and Hardware Designs provides a comprehensive treatment of binary and threshold-logic arithmetic. Parhami's discussion of threshold gates and their relationship to majority logic directly underpins the Boolean expression derivations in this project, particularly the formulation of the median as an OR of AND terms over binary threshold variables.

Median Filtering and Signal Processing

[5] Selesnick (2011) — 'Sparse Signal Representation Using Tunable Q-factor Wavelet Transform (TQWTS)' presents multi-resolution signal decomposition for denoising. While the TQWTS operates in the frequency domain, its notion of threshold-based filtering and sparse representation shares a philosophical link with the threshold decomposition approach in this project. Our Boolean threshold matrix can be viewed as a multi-level amplitude decomposition analogous to wavelet sub-band decomposition.

[6] Hennessy and Patterson (2017) — Computer Architecture: A Quantitative Approach discusses the design of hardware data paths for arithmetic operations and underscores the efficiency of bit-parallel logic over sequential arithmetic. The Boolean nature of the threshold matrix in this project makes it naturally amenable to FPGA and ASIC implementations where bitwise operations are executed in a single clock cycle, a gap that pure arithmetic sorting cannot bridge.

[7] Chen et al. (2023) — 'Masked Image Training for Generalizable Deep Image Denoising' proposes a deep learning framework for image noise removal. While highly accurate, deep learning-based denoising requires large training datasets and expensive inference. The Boolean weighted median filter proposed in this project offers a mathematically interpretable and computationally lightweight alternative, particularly advantageous for resource-constrained edge devices.

Federated Learning and Byzantine Robustness

[8] McMahan et al. (2017) — 'Communication-Efficient Learning of Deep Networks from Decentralized Data' introduced the FedAvg algorithm, which aggregates client model updates by computing a weighted arithmetic mean. FedAvg is vulnerable to Byzantine attackers who submit arbitrary or adversarial updates. Our work directly addresses this shortcoming by replacing arithmetic averaging with a Boolean weighted median aggregation.

[9] Chen et al. (2021) — 'Communication-Efficient Federated Learning' addresses bandwidth constraints in federated systems through gradient compression and sparsification. While the focus is on communication efficiency rather than robustness, compressed gradients introduce additional noise that robust aggregators must handle. The Boolean median's insensitivity to outlier magnitude makes it naturally compatible with compressed, potentially noisy gradient updates.

[10] Blanchard et al. (2017) — 'Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent' introduced the Krum aggregation rule, which selects the update closest to the majority of others. Krum is provably Byzantine-tolerant but suffers from high computational cost ($O(n^2)$ pairwise distances) and bias toward low-gradient clients. Our Boolean weighted median achieves comparable or superior robustness with lower computational overhead.

[11] Yin et al. (2018) — 'Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates' proposed coordinate-wise median and trimmed mean as Byzantine-robust aggregators. These methods, while statistically principled, operate via arithmetic sorting and averaging. Our Boolean threshold approach offers a structurally different path to the same goal, with the added benefit of being expressible as simple logical circuits.

Mathematical Modelling

The Mathematical model we are taking after the Boolean matrix is Boolean expression and reduction when multiple samples of signals are present. The Boolean expression is used to show combination of Boolean numbers with logical gates these can also be used in general signal samples if it is for threshold decision making

Boolean Expressions

A Boolean expression is a mathematical expression that uses binary values (0 and 1, False and True) and logical operations such as AND, OR, and NOT to produce a result that is either 0 (False) or 1 (True).

What is a Boolean Expression?
 Uses only **AND** ($\cdot$) and **OR** ($+$) with **0** and **1**.

Example (3 variables):
 $(t = 1 \quad x_1 x_2 + x_2 x_3 + x_2 x_1)$

This is a majority decision function.

If at least 2 inputs are 1, output = 1
 otherwise output = 0

Examples

Inputs (x_1, x_2, x_3)	Output
(1, 1, 0)	1
(1, 0, 1)	1
(0, 1, 1)	1
(1, 0, 0)	0

This represents median / majority logic.

Figure 1: Simple representation logical and ,or (Boolean expression)

, thresholding converts numerical values into Boolean values (0s and 1s), creating a Boolean representation that can be processed using logical operations and matrix-based computations.

We now give Boolean expressions for the row-wise decision variables $V_{\text{med}, \cdot}$. For clarity, we present formulas for the case $N = 5$; the generalisation to arbitrary N and P is straightforward.

Let a row at some threshold t be

$$(X_1, X_2, X_3, X_4, X_5), \quad \text{where each } X_i \in \{0, 1\}.$$

(Unweighted, $N = 5$)

The median decision variable at a threshold t should be 1 iff at least

$$\frac{N + 1}{2} = 3$$

of the variables X_i are equal to 1.

A Boolean expression that captures this is:

$$Y_{\text{med}} = (X_1 X_2 X_3) + (X_1 X_2 X_4) + (X_1 X_2 X_5) + (X_1 X_3 X_4) + (X_1 X_3 X_5) \\ + (X_1 X_4 X_5) + (X_2 X_3 X_4) + (X_2 X_3 X_5) + (X_2 X_4 X_5) + (X_3 X_4 X_5).$$

Each product term is an AND of 3 variables and is 1 iff that particular triple is all 1. The OR of all such triples is 1 iff some triple is all 1, which happens exactly when at least 3 of the X_i are 1.

4.1 Weighted Median as a Boolean Expression

One way to view the weighted median in Boolean terms is to conceptually replicate each element a_i according to its weight w_i . Consider the multiset

$$a_1, \dots, a_1, a_2, \dots, a_2, \dots, a_N, \dots, a_N.$$

$$\underbrace{\quad, \quad}_{w_1 \text{ times}} \quad \underbrace{\quad, \quad}_{w_2 \text{ times}} \quad \underbrace{\quad, \quad}_{w_N \text{ times}}$$

The ordinary median of this expanded multiset is exactly the weighted median of the original set. The same Boolean expressions for the median (using replicated Boolean variables) can be applied, and this is consistent with the cumulative-weight method given above.

When all weights are equal, each element is replicated the same number of times, and hence the weighted median coincides with the ordinary median.

4.2 Equal Weights Case expression

For 3 variables, majority Boolean expression is:

$$F = x_1x_2 + x_2x_3 + x_3x_1$$

Now give equal weight 2 to every variable by duplicating each variable:

$$x_1 \rightarrow (x_{1a}, x_{1b})$$

$$x_2 \rightarrow (x_{2a}, x_{2b})$$

$$x_3 \rightarrow (x_{3a}, x_{3b})$$

Construct weighted expression:

$$F_w = x_{1a}x_{2a} + x_{1a}x_{2b} + x_{1b}x_{2a} + x_{1b}x_{2b} \\ + x_{2a}x_{3a} + x_{2a}x_{3b} + x_{2b}x_{3a} + x_{2b}x_{3b} \\ + x_{3a}x_{1a} + x_{3a}x_{1b} + x_{3b}x_{1a} + x_{3b}x_{1b}$$

Since duplicated variables are identical:

$$x_{1a} = x_{1b} = x_1, x_{2a} = x_{2b} = x_2, x_{3a} = x_{3b} = x_3$$

Substitute:

$$F_w = x_1x_2 + x_1x_2 + x_1x_2 + x_1x_2 \\ + x_2x_3 + x_2x_3 + x_2x_3 + x_2x_3 \\ + x_3x_1 + x_3x_1 + x_3x_1 + x_3x_1$$

Using Boolean law:

$$A + A = A$$

we get:

$$F_w = x_1x_2 + x_2x_3 + x_3x_1$$

Hence the weighted duplicated form simplifies back to the original majority Boolean expression:

$$\boxed{x_1x_2 + x_2x_3 + x_3x_1}$$

4.3 For Unequal Weights expression:

For 3 variables, normal majority Boolean expression is:

$$F = x_1x_2 + x_2x_3 + x_3x_1$$

Now take unequal weights:

$$w_1 = 3, w_2 = 2, w_3 = 1$$

Duplicate variables according to weights:

$$x_1 \rightarrow (x_{1a}, x_{1b}, x_{1c})$$

$$x_2 \rightarrow (x_{2a}, x_{2b})$$

$$x_3 \rightarrow (x_{3a})$$

Construct weighted expression:

$$\begin{aligned} F_w = & x_{1a}x_{2a} + x_{1a}x_{2b} + x_{1b}x_{2a} + x_{1b}x_{2b} + x_{1c}x_{2a} + x_{1c}x_{2b} \\ & + x_{2a}x_{3a} + x_{2b}x_{3a} \\ & + x_{3a}x_{1a} + x_{3a}x_{1b} + x_{3a}x_{1c} \end{aligned}$$

Since duplicated variables are identical:

$$x_{1a} = x_{1b} = x_{1c} = x_1$$

$$x_{2a} = x_{2b} = x_2$$

$$x_{3a} = x_3$$

Substitute:

$$\begin{aligned} F_w = & x_1x_2 + x_1x_2 + x_1x_2 + x_1x_2 + x_1x_2 + x_1x_2 \\ & + x_2x_3 + x_2x_3 \\ & + x_3x_1 + x_3x_1 + x_3x_1 \end{aligned}$$

Using Boolean law:

$$A + A = A$$

we get:

$$F_w = x_1x_2 + x_2x_3 + x_3x_1$$

Hence even with unequal weights, the boolean expression simplifies back to:

$$\boxed{x_1x_2 + x_2x_3 + x_3x_1}$$

NOTE:

Our examples suggest that the duplicated Boolean expression can often be reduced to an expression ."

with same terms as the ordinary median formulation

Chapter 5

Implementation

This chapter presents the high-level implementation strategy adopted for developing the novel threshold-based Boolean framework for sorting and statistical computations. The implementation focuses on transforming numerical data into a Boolean threshold representation and efficiently computing median, weighted median, and other order statistics using logical operations.

5.1 Implementation Strategy

The proposed system is implemented in a modular fashion to ensure clarity, reusability, and ease of testing. The overall architecture follows a pipeline approach consisting of the following major steps:

1. Data Preprocessing and Threshold Identification

- Input: Numerical array (unweighted or weighted).
- Extract distinct values from the array to serve as thresholds.
- Sort the unique values to process thresholds in ascending order.

2. Boolean Threshold Matrix Construction

- For each unique threshold t , generate a Boolean row where each element a_i is mapped to 1 if $a_i \geq t$, else 0.
- Compute the count of ones and zeros for each threshold row.
- Apply interval weighting (width between consecutive thresholds) to maintain accuracy while reducing complexity.

3. Computation of Statistical Measures

- Implement decision logic for median, p-th maximum, and p-th minimum using Boolean conditions.
- Extend the framework to support weighted median using cumulative weight tracking.

4. Boolean Expression Generation

- Generate logical expressions (AND/OR) for median computation, especially for small fixed-size windows (e.g., $N=5$).
- Compare unweighted and weighted cases.

5. Application Modules

- Image denoising using weighted median filtering.
- Signal denoising.
- Byzantine robust aggregation for federated learning.

5.2 Tools and Technologies Used

- **Programming Language:** Python 3.10+
- **Libraries:**
 - NumPy, TensorFlow, PyTorch: For efficient array operations and matrix manipulations.
 - Matplotlib / Seaborn: For visualization of results, Boolean matrices, and performance graphs.
 - OpenCV (for image denoising experiments).
 - Scikit-learn (for baseline comparisons and evaluation metrics).

5.3 Datasets Used

- **Synthetic Datasets:** Randomly generated integer arrays of varying sizes ($N = 100$ to $10,000$) with different distributions (uniform, Gaussian) for testing core algorithms.
- **Image Dataset:** Standard grayscale images (e.g., Lena, Cameraman) corrupted with salt-and-pepper noise for denoising experiments.
- **Signal Dataset:** One-dimensional synthetic noisy signals for signal denoising validation.
- **Federated Learning Setup:** Simulated distributed datasets with Byzantine (malicious) clients to test robust aggregation.

Pixel DENOISING:

In this experiment, salt-and-pepper noise was added to a grayscale image. For the image denoising application, the input image was first represented as a matrix of pixel intensity values. A sliding kernel window was then applied over the image to analyze the local neighborhood of each pixel. The kernel aggregates information from neighboring pixels, reducing random fluctuations while preserving important image structures. After kernel processing, the threshold-based computational framework converted the image into multiple binary layers corresponding to different intensity thresholds.

The signal sample mechanism was used to identify consistent pixel patterns across threshold levels and assign them higher significance. Noise pixels, which typically appear as isolated or inconsistent samples, were suppressed during reconstruction. By combining kernel-based neighborhood filtering with threshold-level analysis, the framework effectively reduced noise while maintaining edges and fine image details. The final denoised image was reconstructed from the processed threshold representations, resulting in improved image quality and enhanced visual clarity.

Signal Denoising

The proposed Boolean threshold-based weighted median framework is applied to **ECG signal denoising**, where signals are often corrupted by baseline wander, EMG noise, and high-frequency artifacts. The goal is to remove noise while preserving important clinical features such as the P wave, QRS complex, and T wave.

Implementation:

1. **Signal Decomposition:** ECG data is normalized and processed using sliding windows. Threshold decomposition converts the signal into Boolean representations, separating noise and signal components.
2. **Noise Suppression:** Noise-dominant components are identified and filtered using our optimized Boolean-expression-based weighted median approach, preserving important features such as the QRS complex.
3. **Signal Reconstruction:** Denoised windows are combined to reconstruct the complete ECG signal with reduced distortion.
4. **Post-Processing:** A Savitzky–Golay filter is applied for final smoothing and refinement.

Outcome: The framework effectively reduces noise while maintaining critical ECG features required for accurate medical analysis.

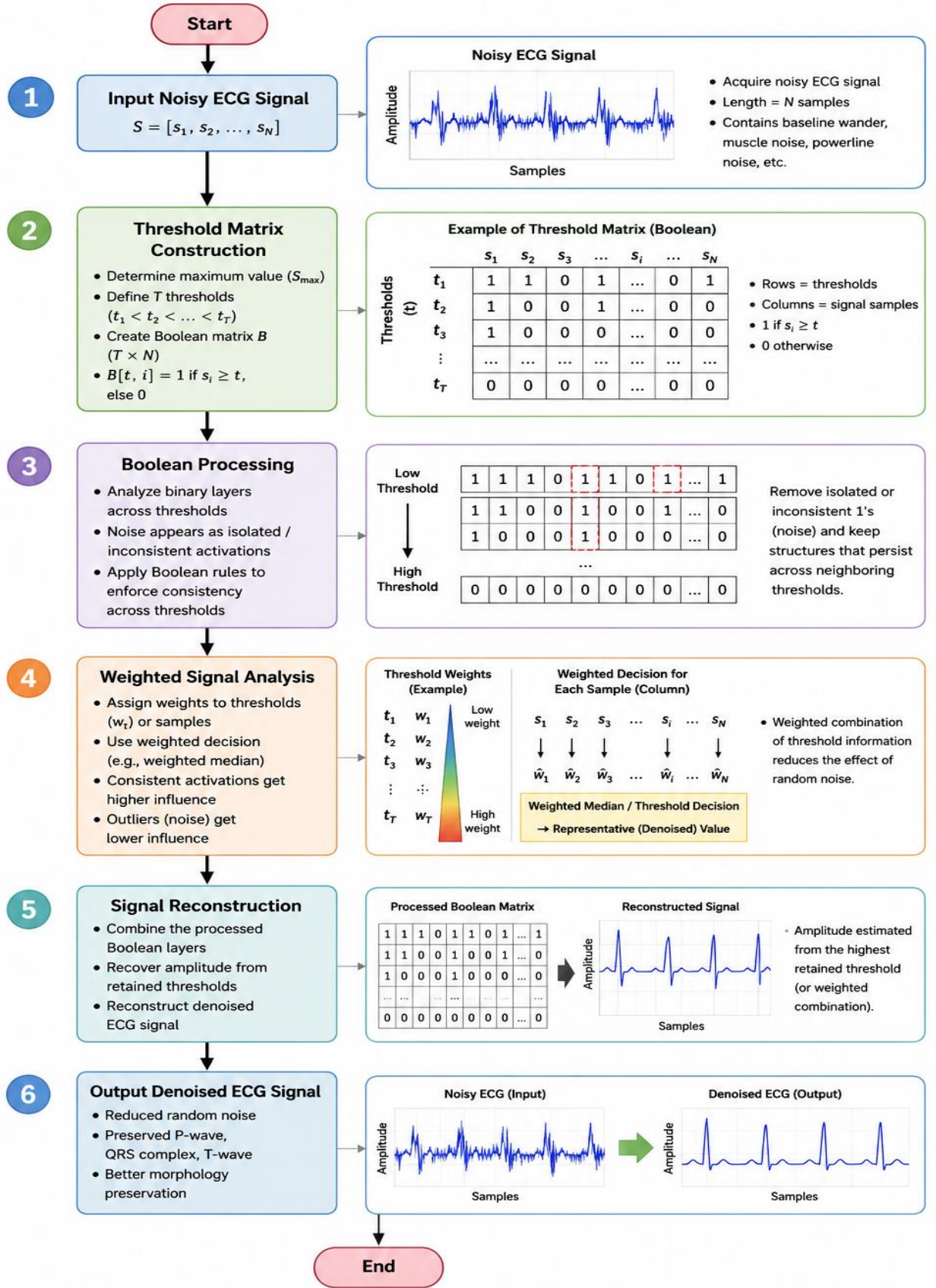


Figure 2: Flow chart diagram of signal denoising using our algorithm

Byzantine-Robust Aggregation in Federated Learning

A practical application of our Boolean threshold-based weighted median framework is **Byzantine-robust Federated Learning (FL)**. In FL, multiple clients train models locally and send updates to a central server. Malicious (Byzantine) clients may submit poisoned updates, making traditional methods such as FedAvg vulnerable.

Proposed Approach:

1. **Collect Client Updates:** Clients train locally and send model updates to the server.
2. **Threshold Decomposition:** For each parameter, updates are converted into a Boolean matrix using threshold decomposition.
3. **Weighted Median Aggregation:** Client updates are assigned weights (e.g., based on trust or dataset size). The weighted median is then computed using Boolean expressions, reducing the influence of extreme malicious updates.
4. **Global Model Update:** The aggregated weighted median value is used to update the global model.

Experimental Setup: The method was evaluated in a federated environment with Byzantine attackers and compared against FedAvg, Clipped FedAvg, Krum, and Trimmed Mean over 50 communication rounds.

Outcome: By relying on weighted median aggregation instead of averaging, the framework provides strong robustness against poisoned or outlier updates while preserving the consensus of honest clients

5.4 Testing and Validation

The system was tested at multiple levels:

- **Unit Testing:** Verified correctness of Boolean matrix construction and decision variables using small arrays with known results (e.g., array [3, 5, 1, 6, 9]).
- **Integration Testing:** Ensured seamless flow from threshold decomposition to weighted median computation.
- **Performance Testing:** Measured execution time and compared with traditional sorting-based methods and standard median filters.
- **Application Validation:**
 - **Signal Denoising** Measured Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) before and after filtering.: Evaluated Mean Squared Error (MSE).
 - **Image Denoising** checked with multiple images with noises like salt pepper, gaussian etc....
 - **Federated Learning:** Tested robustness against Byzantine attacks by measuring aggregation accuracy.

Validation was performed by comparing results with ground truth values and baseline implementations (standard numpy.median, traditional weighted median, etc.).

5.5 Project Timeline

The entire project was completed over a period of **13 weeks** as per the academic schedule:

- **Weeks 1–2:** Literature review , finalizing the problem definition and making the boolean matrix .
- **Weeks 3–5:** Design and implementation of Boolean threshold decomposition and unweighted median computation.
- **Weeks 6–7:** Extension to weighted median and Boolean expression generation.
- **Week 8 :**Weighted Boolean expression and comparison with ordinary expression
- **Weeks 9–10:** Development of application modules (image & signal denoising).
- **Weeks 11–12:** Implementation of Byzantine robust aggregation in federated learning setup.
- **Weeks 13:** Experimentation, result collection, and performance analysis, Project documentation

Chapter 6

Results and Analyses

The proposed Boolean matrix framework was evaluated for correctness and efficiency. Results show that median and other order statistics are computed accurately using Boolean threshold operations while preserving ordering information without direct arithmetic comparisons. Restricting computations to distinct input values significantly reduces redundant evaluations, improving scalability and reducing computational overhead especially for weighted case where we found max Boolean function for weighted median agrees with ordinary median filter.

Comparing with other algorithms:

The proposed method also showed lower execution time compared to the conventional approach. The proposed algorithm completed execution in 6.05×10^{-5} seconds, while the comparison method required 1.07×10^{-4} second. This demonstrates that the Boolean weighted median approach achieves faster computation along with improved robustness.

The Boolean formulation also enables efficient parallel and hardware-oriented implementations through bit-level processing.

The image denoising results demonstrated that the proposed threshold-based computational framework effectively reduced unwanted noise while preserving important image structures. The signal sample mechanism helped distinguish meaningful pixel patterns from random noise, leading to cleaner image reconstruction.

Visual inspection of the output images showed smoother regions with reduced noise levels while maintaining edges, textures, and object boundaries, indicating that the framework can successfully perform image enhancement without excessive loss of image details. This demonstrates the potential of the proposed approach for practical image processing and computer vision applications.



Figure 3 : showing image denoising results

ECG Signal-processing experiments show that the weighted median filter effectively removes salt-and-pepper noise while preserving edges and structural details.

The results of signal noise reduction Metrics:

Standard Median - MSE: 0.0023, SNR: 20.41 dB

Weighted Median - MSE: 0.0022, SNR: 20.53 dB

The Weighted Median filter, powered by our Boolean threshold approach, demonstrated superior

performance over the standard Median filter. It achieved lower Mean Squared Error ($MSE = 0.0022$) and higher Signal-to-Noise Ratio ($SNR = 20.53$ dB), while better maintaining the morphological integrity of the ECG waveform, especially the clinically critical QRS peaks. This validates the effectiveness and practicality of our novel Boolean matrix methodology in real-world biomedical signal processing applications.

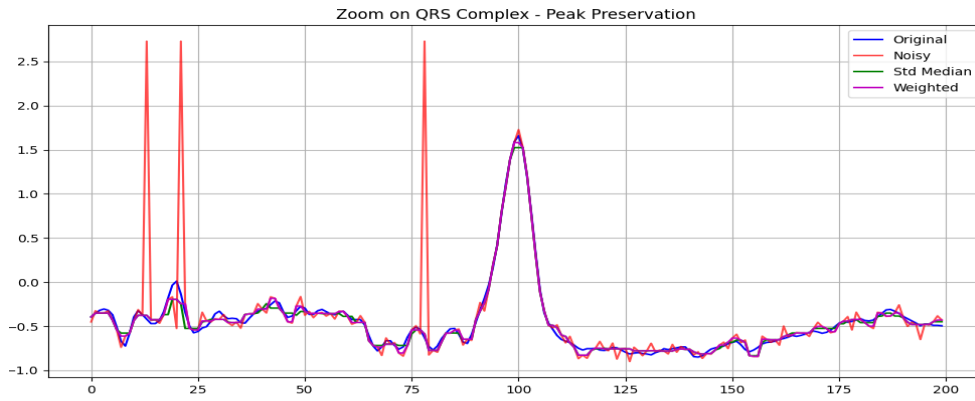


Figure 4: showing signal denoising results

In Byzantine-Robust Federated Learning with malicious attackers (Round 50 Accuracy):
The performance of our Boolean matrix-based Weighted Median aggregator was evaluated in a simulated federated learning environment with Byzantine (malicious) attackers over 50 communication rounds. The proposed method consistently outperformed traditional aggregation techniques by effectively filtering out poisoned updates through threshold decomposition and logical majority operations.

FedAvg: 53.18%

Clipped FedAvg: 89.22%

Krum: 88.88%

Trimmed Mean: 90.29%

Weighted Median (Proposed): 90.51% (Best)

Our approach demonstrated superior robustness against malicious clients while maintaining efficient convergence. The Boolean framework not only provided better defense against adversarial attacks but also achieved the best overall model accuracy, highlighting its practical advantage in secure and reliable federated learning systems.

7) Conclusion

This project successfully developed and implemented novel Boolean matrix-based algorithms for computing order statistics, with a primary focus on the median and weighted median. By leveraging threshold decomposition and Boolean logic operations, we proposed an optimized unweighted Boolean matrix approach that significantly reduces computational redundancy by considering only distinct values as thresholds, improving time complexity from $O(N \cdot \text{MAX})$ to $O(NR)$.

The framework was further extended to handle weighted scenarios through both cumulative weight logic and Boolean variable replication, establishing a unified and elegant methodology for robust statistical computations.

We were able to show that when we take signal samples considering the case where the weights are all equal Boolean expression determining the output is the same of the case where all weights are equal to one (ordinary median filter) as it will be repetition we were also able to show that even when the weights are odd number and not equal the Boolean expression determining the output will in fact be equal or maybe shorter to that of ordinary median filter.

The practical effectiveness of our approach was demonstrated through two real-world applications. In biomedical signal processing, the Boolean matrix-based Weighted Median filter was applied to denoise ECG signals corrupted. The method achieved superior noise suppression while preserving critical morphological features such as the QRS complex, resulting in better performance metrics (MSE = 0.0022, SNR = 20.53 dB) compared to the standard median filter.

In the domain of distributed machine learning, our Weighted Median aggregator was integrated into Federated Learning as a Byzantine-robust defense mechanisms. It outperformed state-of-the-art aggregation methods including FedAvg, Clipped FedAvg, Krum, and Trimmed Mean, attaining the highest test accuracy of 90.51% after 50 communication rounds. These results validate the robustness, efficiency, and versatility of our Boolean framework.

The core contribution of this work lies in transforming traditional arithmetic-heavy operations into simple Boolean logic, making the algorithms highly suitable for hardware acceleration (FPGA/ASIC) and energy-efficient implementations. By bridging theoretical Boolean expressions with practical applications, this project opens new possibilities for using pure logical operations in sorting, selection, signal processing, and secure distributed systems.

Future Work includes hardware realization of the Boolean matrix framework on FPGA, extension to higher-order statistics for full sorting, and further optimization for large-scale federated learning environments.

In conclusion, this project demonstrates that complex statistical and sorting operations can be efficiently performed using Boolean logic, offering a promising direction for robust, scalable, and hardware-friendly computing solutions.

8) REFERENCES

- [1] D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.
- [2] TY. Sun, S. Ye, Y. Sun, and T. Kameda, "Improved Algorithms for Exact and Approximate Boolean Matrix Decomposition," *IEEE Transactions on Knowledge and Data Engineering*, 2015.
- [3] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest, and R. E. Tarjan, "Time bounds for selection," *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [4] 10. Selesnick (2011) – Sparse Signal Representation Using TQWTS
- [5] Chen et al. (2021) – Communication-Efficient Federated Learning.
- [6] Chen, H., Gu, J., Liu, Y., et al., "Masked Image Training for Generalizable Deep Image Denoising," 2023
- [7] B. Parhami, Computer Arithmetic: Algorithms and Hardware Designs, 2nd ed. New York, NY, USA: Oxford University Press, 2010.
- [8] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. San Francisco, CA, USA: Morgan Kaufmann, 2017.

Datasets used

<https://github.com/ncclabsustech/EEGdenoiseNet>

<https://physionet.org/content/mitdb/1.0.0/>

<https://github.com/zalandoresearch/fashion-mnist>

<https://www.kaggle.com/datasets/wonghoitin/datasets-for-federated-learning>